

ARBEITSTHEMA / TITEL

UNTERTITEL

NAME:

VORNAME:

MATRIKEL-NR.:

STUDIENGANG:

E-MAIL:

SEMINAR:

DOZENTIN:

LEHRSTUHL:

INSTITUT:

FAKULTÄT:

MODUL:

FACHSEMESTER:

ABGABEDATUM:

INHALTSVERZEICHNIS

I Die Praktikumsstelle - Formale Bedingungen	1
II Aufgaben und Tätigkeiten - Der Ablauf des Praktikums	2
2.1 Umfeld der Praktikumsstätigkeit	2
2.2 Aufgabe und Ziel	2
2.3 Tätigkeiten und Arbeitsergebnisse	3
2.4 Fachspezifische Reflexion über das Praktikum	7
III Fazit und Bewertung	8
IV Anhang	10
V Literaturverzeichnis	11
VI Eidesstattliche Versicherung	12

I DIE PRAKTIKUMSSTELLE - FORMALE BEDINGUNGEN

- Genaue Beschreibung der Institution, Was macht die Institution? Wie groß usw.? Welche Aufgabe hat meine Abteilung? Leitungs- / Organisationsstruktur?
- Wie habe ich die Praktikumsstelle gefunden (Bewerbung, Kontaktaufnahme)? Welche Erfahrungen gewinne ich aus dem Vorgehen bei der Bewerbung?
- Wie sind die Arbeitsbedingungen? Gab es Fahrtkostenzuschuss, Aufwandsentschädigung, sonstige Unterstützung? Wie waren die Arbeitszeiten?

II AUFGABEN UND TÄTIGKEITEN - DER ABLAUF DES PRAKTIKUMS

2.1 UMFELD DER PRAKTIKUMSTÄTIGKEIT

Im Praktikum habe ich an zwei unterschiedlichen Softwareprojekten gearbeitet. Ein Teil meiner Arbeit betraf ein internes Erasmus-Portal für die Verwaltung von Staff-Mobility-Fällen. In den Projektdateien ist zu sehen, dass die Anwendung als Webanwendung aufgebaut wurde und Bereiche für Staff, Officer und Admin enthält. Damit ging es nicht nur um einzelne Seiten, sondern um einen zusammenhängenden Verwaltungsablauf mit Formularen, geschützten Bereichen, Datenmodell und Rollenlogik.

Schon an der Navigation lässt sich erkennen, dass das Portal nicht nur aus einer einzelnen Antragsmaske bestand. Es gibt eine Startseite, Login und Registrierung, eine Statusseite für den lokalen Betrieb und danach getrennte Dashboard-Bereiche. Staff arbeitet mit Profil und eigenen Fällen, Officer mit Review-Ansichten und Filtern, und Admin mit Benutzerverwaltung, Stammdaten und Audit-Log. Dadurch hatte das Projekt eher den Charakter eines kleinen internen Verwaltungssystems als den einer einfachen Website.

Daneben habe ich an einer Android-App gearbeitet, mit der man Deutsch mit Karten lernen kann. Diese Anwendung ist als reine Android-App aufgebaut und arbeitet lokal, also ohne Server, Benutzerkonto oder Login. Nach dem Start landet man direkt in einer Kategorienansicht und wechselt von dort in Decklisten und Lernsitungen. Dazu kommen eine Statistikseite, gemischte Lernsitungen und ein seitliches Menü für Einstellungen. Im Code ist außerdem klar getrennt, was zur UI, zur Fachlogik und zur Datenhaltung gehört.

Die App wirkte nach außen kleiner als das Portal, war im Code aber trotzdem ziemlich vielschichtig. Es gibt nicht nur die eigentlichen Screens, sondern auch ViewModels, Use Cases, lokale Datenhaltung mit Room, Einstellungen über DataStore und eine ganze Reihe von Tests. Gerade weil die App ohne Backend auskommt, musste vieles direkt auf dem Gerät sauber gelöst werden. Das betrifft zum Beispiel die Frage, wann Karten wieder fällig sind, wie Zähler aktuell bleiben und welche Einstellungen nach einem Neustart noch vorhanden sind.

Dadurch war das Praktikum insgesamt recht abwechslungsreich. Beim Webprojekt standen eher Rollen, Freigaben, Verwaltungsabläufe und serverseitige Logik im Vordergrund. Bei der App ging es stärker um Lernfluss, lokale Speicherung und Zustände auf dem Gerät selbst. In beiden Projekten musste ich aber über die Oberfläche hinausdenken und immer auch den Ablauf im Hintergrund mitberücksichtigen.

2.2 AUFGABE UND ZIEL

Meine Aufgabe war es, beide Projekte Schritt für Schritt zu einer brauchbaren Form weiterzuentwickeln. Beim Erasmus-Portal hieß das, Anforderungen aus einem internen Verwaltungsprozess in konkrete Funktionen zu übersetzen. Aus den Dateien geht hervor, dass dafür öffentliche Seiten wie Login und Registrierung, geschützte Bereiche für die verschiedenen Rollen, serverseitig abgesicherte Abläufe sowie ein Datenmodell für Benutzer, Mobilitätsfälle, Dokumente, Statusverläufe und Berichte aufgebaut wurden. Es ging also nicht nur um einzelne Formulare, sondern um einen vollständigen Ablauf von der Antragstellung bis zur Prüfung und Verwaltung.

Ein Teil der Aufgabe lag deshalb darin, eher allgemeine Anforderungen zuerst technisch greifbar zu machen. Wenn Departments zu Fakultäten gehören, Dokumente privat bleiben sollen

oder Statuswechsel später nachvollziehbar sein müssen, reicht eine sichtbare Seite allein nicht aus. Dann braucht man auch passende Relationen im Datenmodell, serverseitige Prüfungen in den API-Routen und eine Struktur, in der sich Entwurf, Einreichung, Dokumentenprüfung und Archivierung sauber auseinanderhalten lassen. Außerdem sollte der Stand früh lokal startbar und testbar sein, damit neue Teile nicht nur im Code, sondern im laufenden System geprüft werden konnten.

Bei der Android-App lag das Ziel anders. Dort sollte aus einem ersten Gerüst eine nutzbare Lernanwendung werden, mit der man direkt arbeiten kann. Nach dem Start sollte man eine Kategorie auswählen, ein Deck öffnen, Karten aufdecken, bewerten und danach sinnvoll weitermachen können. Gleichzeitig musste der Lernstand dauerhaft gespeichert werden. Dazu kamen Einstellungen wie Themes, Audio und zusätzliche Übersetzungshilfen sowie kleinere gestalterische Aufgaben wie Hintergründe und das App-Icon.

Mir war bei der App außerdem wichtig, dass sie nicht wie eine lose Sammlung einzelner Karten wirkt. Schon auf den ersten Seiten sollte erkennbar sein, welche Themen es gibt, wo Wiederholungen anstehen und wie viele neue Karten für den Tag noch sinnvoll sind. Die App sollte also nicht einfach alle Inhalte auf einmal zeigen, sondern Lernen in kleinen, nachvollziehbaren Schritten organisieren. Daraus ergaben sich viele technische Aufgaben, obwohl die Anwendung im Vergleich zum Webprojekt auf den ersten Blick einfacher aussieht.

Ein weiterer Punkt war der Startzustand der App. Da es keinen Login, keinen Server und keinen externen Datenabruf gibt, musste die Anwendung ihren Ausgangszustand selbst mitbringen. Im Projekt ist deshalb eine Seed-Datei vorhanden, die beim ersten Start Kategorien, Decks, Karten und anfängliche Review-Zustände anlegt. Dadurch war die Arbeit nicht nur auf Logik und Oberfläche beschränkt, sondern auch darauf, wie Inhalte vernünftig in die App gelangen und danach dauerhaft weiterverarbeitet werden.

Gemeinsam war beiden Projekten, dass die sichtbare Oberfläche nur ein Teil der Arbeit war. Ich musste immer gleichzeitig darauf achten, wie Screens oder Seiten zusammenhängen, wie Daten gespeichert werden, wie Zustände weitergegeben werden und wie sich die Anwendung in typischen und untypischen Fällen verhält. Gerade dadurch wurde aus einzelnen Bausteinen nach und nach eine wirklich benutzbare Anwendung.

2.3 TÄTIGKEITEN UND ARBEITSERGEBNISSE

Ein größerer Teil meiner Arbeit lag im Webprojekt. Zunächst musste eine Grundlage stehen, auf der sich die Anwendung lokal starten, testen und weiterentwickeln ließ. Im Projekt sind dafür die eigentliche Webanwendung, Konfigurationsdateien, Skripte für Entwicklung und Build, Datenbankankündigung, Seed-Daten und Testaufbau vorhanden. Dadurch war die Anwendung nicht nur als Quellcode da, sondern in einem Zustand, in dem sich ganze Abläufe lokal nachvollziehen ließen.

Zu dieser Grundlage gehörte nicht nur der eigentliche Anwendungscode. Im Projekt liegen auch Umgebungsvariablen, Migrationsskripte, Seed-Daten, Demo-Konten, eine Statusseite und Dokumentationen für den lokalen Start. Für meine Arbeit war das wichtig, weil ich neue Teile immer wieder in einem laufenden System gegenprüfen konnte, statt sie nur isoliert zu bauen.

Danach ging es um Benutzerkonten, Rollen und Freigaben. Im Repository ist zu sehen, dass neue Staff-Nutzer sich registrieren können, dann aber zunächst auf eine Freigabe warten. Officer-

und Admin-Bereiche sind getrennt davon abgesichert. Dazu kamen Profilseiten und Stammdatenbereiche, in denen zum Beispiel Fakultäten, Departments, akademische Jahre, Statuswerte oder Upload-Einstellungen verwaltet werden. An solchen Stellen merkt man gut, dass fachliche Anforderungen zuerst in ein sauberes Datenmodell übersetzt werden mussten, bevor die Oberfläche sinnvoll funktionieren konnte.

Praktisch war dabei oft wichtig, dieselben Daten aus verschiedenen Blickwinkeln zu behandeln. Staff sieht nur den eigenen Bereich und die eigenen Fälle. Officer braucht Such- und Prüffunktionen über alle eingereichten Vorgänge hinweg. Admin bearbeitet Benutzer, Stammdaten und Einstellungen. Dadurch musste ich nicht nur Seiten bauen, sondern auch serverseitig absichern, wer welche Daten sehen oder verändern darf.

Viel Arbeit steckte außerdem in den eigentlichen Mobilitätsfällen. Staff-Nutzer können mehrere Fälle anlegen, als Entwurf speichern, später weiterbearbeiten und schließlich einreichen. Im Projekt ist zu sehen, dass Entwürfe und Einreichungen unterschiedlich behandelt werden und dass Statuswechsel in einer eigenen Historie gespeichert werden. Später kamen Dokumenten-Upload, geschützte Download-Routen, Kommentierung, Prüfung durch Officer, Archivierung und serverseitige Auswertungen dazu. Im weiteren Verlauf wurden auch Fehlermeldungen, Validierungstexte, Tabellen, Statuskennzeichnungen und leere Zustände überarbeitet. Dadurch ist aus einzelnen Verwaltungsseiten ein zusammenhängender Arbeitsablauf geworden.

Bei den Mobilitätsfällen war außerdem wichtig, dass der Ablauf nicht zu starr wird. Ein Fall kann zuerst unvollständig als Entwurf beginnen und erst später vollständig eingereicht werden. Deshalb gibt es im Projekt getrennte Validierungsstufen und klare Regeln dafür, wann ein Statuswechsel überhaupt erlaubt ist. Gerade solche Unterschiede wirken in der Oberfläche klein, entscheiden aber darüber, ob ein Formular im Alltag brauchbar ist oder schnell im Weg steht.

Besonders deutlich wurde das bei den Dokumenten. Es reichte nicht, einfach einen Upload-Button einzubauen. Die Dateien sollten privat bleiben, versioniert werden und nur über geschützte Routen herunterladbar sein. Gleichzeitig musste sauber getrennt bleiben, was der aktuelle Stand eines Falls ist und was nur den Review-Zustand eines Dokuments betrifft. Genau an solchen Stellen wurde aus einer normalen Webanwendung eher ein Verwaltungswerkzeug mit nachvollziehbaren Regeln.

Später ging es nicht mehr nur um einzelne Vorgänge, sondern auch um den Überblick über den Gesamtstand. Officer und Admin können Fälle suchen, filtern, kommentieren, archivieren und in Berichten auswerten. Dazu kamen CSV-Exporte und ein Audit-Log für wichtige Aktionen. Für mich war das ein anderer Arbeitsschritt als am Anfang, weil ich nicht mehr nur auf einzelne Formulare geschaut habe, sondern darauf, wie das System mit vielen Fällen und verschiedenen Rollen zusammenhängt.

Ein weiterer Teil der Website-Arbeit bestand aus Überarbeitung und Absicherung. Im Repo ist zu sehen, dass neben Unit- und Integrations-Tests auch ein größerer End-to-End-Ablauf vorhanden ist, der Registrierung, Freigabe, Profilbearbeitung, Fallanlage, Upload, Review, Archivierung und Reporting zusammennimmt. Dazu kamen kleinere Korrekturen an Fehlermeldungen, leeren Zuständen, Tabellen, Statuskennzeichnungen und der allgemeinen Oberfläche. Gerade an diesen Stellen habe ich gemerkt, dass ein Projekt nicht nur durch neue Funktionen wächst, sondern auch durch wiederholtes Nachbessern.

Zum Ende hin gehörte auch dazu, Setup, Testabläufe, Demo-Konten und Speicherverhalten sauber zu dokumentieren. Das klingt weniger sichtbar als eine neue Seite, war aber wichtig, damit der Stand lokal reproduzierbar bleibt und man nicht jedes Mal wieder bei Null anfängt.

Daneben habe ich an der Android-App gearbeitet, die im Vergleich dazu einen ganz anderen Schwerpunkt hatte. Dort ging es weniger um Rollen und Freigaben, sondern um einen klaren Lernfluss auf dem Gerät selbst. Ein wichtiger Schritt war die Umstellung von einer Fake-Repository-Lösung auf eine lokale Speicherung mit SQLite beziehungsweise Room. Erst dadurch konnte die App Fortschritt, Kartenzustände und Lernprotokolle wirklich speichern und über mehrere Sitzungen hinweg mit echten Daten arbeiten.

Am Commit-Verlauf lässt sich gut erkennen, dass die App nicht in einem Schritt entstanden ist. Am Anfang stand eher ein Framework mit ersten Beispielsichten. Später kamen der Umbau der Architektur, die lokale Datenbank, ein größerer Review-Flow und anschließend viele kleinere Korrekturen dazu. Das war für mich auch im Arbeitsprozess spürbar, weil ich nicht einfach nur neue Teile gebaut habe, sondern frühere Entscheidungen immer wieder anpassen musste, sobald mehr echte Logik dazugekommen ist.

Später habe ich die Struktur des Projekts noch einmal stärker überarbeitet. Im aktuellen Stand sind die Klassen klar in UI, Domain und Datenlogik aufgeteilt. In der Domain-Schicht liegen Modelle und Use Cases für Kartenauswahl, Bewertung und Statistik. In der Daten-Schicht liegen Entities, DAOs, Mapper und die Repository-Implementierung. Diese Trennung war für die weitere Arbeit hilfreich, weil der Code dadurch lesbarer wurde und sich leichter testen ließ.

Auf der sichtbaren Seite musste die Navigation zwischen Kategorienansicht, Deckliste, Lernsitzung, Mixed Study und Statistik aufgebaut und übersichtlich gehalten werden. Dazu kamen gemeinsame Komponenten wie Statistikleisten, Dialoge und das seitliche Menü für Einstellungen. Gleichzeitig steckte viel Arbeit in der Logik hinter dem Lernablauf. Neue und fällige Karten werden unterschiedlich behandelt, für neue Karten gibt es ein Tageslimit, und die Zähler für Due und New werden direkt aus Datenbank und Lernprotokoll berechnet. Auch die nächste Karte wird nicht einfach starr ausgegeben, sondern aus einer kleinen Kandidatenmenge ausgewählt. Daran merkt man gut, dass es bei der App nicht nur um Screens ging, sondern genauso um den Ablauf dahinter.

Gerade die sichtbaren Wege durch die App mussten nach und nach sauber zusammengesetzt werden. In der Kategorienansicht werden Themen mit ihren offenen Wiederholungen und neuen Karten angezeigt. Von dort geht es in die Decklisten, in denen jedes Deck eigene Due- und New-Zähler hat. Per Long-Press lässt sich für ein Deck das Tageslimit neuer Karten anpassen. Daneben gibt es eine eigene Konfigurationsseite für gemischte Sitzungen, auf der Themen, Lernstufen, Modus und neue Karten pro Sitzung ausgewählt werden können. Solche Stellen wirken in der fertigen App recht schlicht, machen in der Umsetzung aber nur dann Sinn, wenn Navigation, Zähler und gespeicherte Einstellungen zusammenpassen.

Auch die eigentliche Lernsitzung war aufwendiger, als es zuerst aussieht. Die Karte wird vorne gezeigt, dann kann die Antwort eingeblendet und anschließend mit Again, Hard, Good oder Easy bewertet werden. Zu den Buttons gehören Intervallhinweise, damit ungefähr sichtbar wird, wann eine Karte wiederkommt. Wenn TTS aktiviert ist, werden deutsche Karten vorgelesen. Für bestimmte Inhalte lassen sich zusätzliche Übersetzungen einblenden. Außerdem muss die Sitzung sauber auf einen Abschlusszustand wechseln, wenn keine Karte mehr sinnvoll in die aktuelle

Runde passt. Gerade bei einer Lern-App merkt man schnell, dass sich kleine Unstimmigkeiten im Ablauf sofort unangenehm anfühlen.

Auf Datenebene musste die App stärker ausformuliert werden, als man es auf den ersten Blick erwartet. Inhalte und Lernfortschritt liegen nicht zusammen in einer einzigen Struktur, sondern sind getrennt in Decks, Karten, ReviewState und StudyLog abgebildet. Das war notwendig, damit sich die Karteninhalte nicht bei jeder Bewertung mitändern und damit man aus den Logdaten später Statistiken und Zähler ableiten kann. Im Projekt ist auch zu sehen, dass die App die täglichen New-Zähler nicht einfach fest speichert, sondern aus dem aktuellen Zustand und den bisherigen Lerneinträgen neu berechnet. Dadurch musste ich mich viel intensiver mit Datenmodellierung und Abfragen beschäftigen, als man bei einer kleinen App vielleicht zuerst denkt.

Der eigentliche Kern der App liegt in der Auswahl und Bewertung der Karten. Neue Karten und fällige Karten werden unterschiedlich behandelt. Nach einer Bewertung wird der nächste Zustand nicht nur auf dem Bildschirm angepasst, sondern in der Datenbank mit neuem Fälligkeitszeitpunkt, Intervall und Wiederholungszahl gespeichert. Dazu kommt, dass die App nicht immer einfach die erste passende Karte nimmt. Stattdessen arbeitet sie mit einer Kandidatenmenge, vermeidet ganz frische Wiederholungen und mischt die Auswahl etwas auf. Das klingt nach einem kleinen Detail, war aber wichtig, damit sich eine Sitzung nicht zu mechanisch oder fehlerhaft anfühlt.

Besonders hilfreich für die Erweiterung der App war die Seed-Struktur. Beim ersten Start werden nicht nur ein paar Platzhalterkarten angelegt, sondern ein größerer Startbestand an Themen wie Travel, Food & Drinks, Shopping, Health oder Emergencies. Innerhalb dieser Themen gibt es mehrere Lernstufen, zum Beispiel Core Words, Übersetzungen, Nomen, Sätze und Kasusübungen. Dadurch konnte ich nicht nur an leeren Strukturen arbeiten, sondern mit echten Inhalten testen, wie sich Zähler, gemischte Sitzungen und Statistik in einem realistischeren Datenbestand verhalten.

Ein weiterer Punkt war die Zustandsverwaltung. Der aktuelle Lernstatus, Lade- und Abschlusszustände einer Sitzung sowie Einstellungen wie Theme, Audio und zusätzliche Übersetzungen mussten nicht nur angezeigt, sondern auch über einen Neustart der App hinweg behalten werden. Diese Werte werden lokal gespeichert und über ViewModels wieder in die Oberfläche gegeben. Gerade bei einer App, die in kurzen Sitzungen benutzt wird, war das wichtig, damit sich der Stand nach einer Unterbrechung nicht willkürlich anders anfühlt.

An diesem Punkt ging es nicht nur um funktionale Dinge, sondern auch um Feinarbeit. Im seitlichen Menü können Theme, TTS und Übersetzungshilfen umgeschaltet werden. Für die Themes gibt es eigene Hintergründe, und auch das Launcher-Icon wurde als eigene Ressource ausgearbeitet. Auf der Statistikseite werden nicht nur rohe Zahlen angezeigt, sondern auch Tageswerte, eine Serie von Lerntagen und eine einfache Wochenansicht. Solche Teile sind für die Kernlogik nicht das Wichtigste, machen aber einen großen Unterschied dabei, ob eine App sich am Ende nur technisch oder auch wirklich benutzbar anfühlt.

Zur Arbeit gehörte deshalb immer wieder auch Debugging und Absicherung. Im Testordner liegen nicht nur allgemeine Beispieltests, sondern Prüfungen für Scheduling, Kartenauswahl, Counter-Logik, Live-Updates, Mixed Sessions und Regressionen. Im Commit-Verlauf sieht man außerdem, dass zwischendurch auch Debug-Hilfen eingebaut und konkrete Probleme im Lernablauf behoben wurden, zum Beispiel bei der Frage, wie schnell Karten nach Again wieder

erscheinen sollen. Gerade bei der App war das wichtig, weil viele Fehler nicht sofort zu einem Absturz führen, sondern eher zu falschen Zählern oder einem komischen Lerngefühl im laufenden Gebrauch.

Rückblickend waren die beiden Projekte inhaltlich unterschiedlich, aber im Arbeitsalltag gab es viele Gemeinsamkeiten. In beiden Fällen musste ich neue Teile umsetzen, bestehende Logik überarbeiten und Randfälle mitdenken. Beim Webprojekt betraf das zum Beispiel Rechte, Statuswechsel und Dokumentversionen. Bei der App waren es eher leere oder abgeschlossene Sitzungen, die erste Initialisierung der Daten und die Frage, ob Zähler und Lernlogik konsistent bleiben. Dazu kamen in beiden Projekten Tests, kleinere UI-Korrekturen, Dokumentation und sichtbare Feinarbeit an Ressourcen und Darstellung.

2.4 FACHSPEZIFISCHE REFLEXION ÜBER DAS PRAKTIKUM

Fachlich war das Praktikum für mich spannend, weil ich an zwei sehr unterschiedlichen Arten von Anwendungen gearbeitet habe. Im Webprojekt ging es stark um Verwaltungslogik. Dort musste ich sehen, wie Frontend, Backend und Datenbank zusammenarbeiten, damit Rollen, Formulare, Status und Dokumente sauber ineinandergreifen. Bei der Android-App stand stärker im Vordergrund, wie eine mobile Anwendung lokal mit Zuständen, Navigation und Datenhaltung umgeht, ohne sich auf einen Server zu stützen.

Beim Webprojekt habe ich besonders gemerkt, wie schnell eine scheinbar einfache Regel mehrere Schichten betrifft. Die Aussage, dass nur freigegebene Staff-Nutzer eigene Fälle bearbeiten dürfen, bedeutet nicht nur eine sichtbare Sperre in der Oberfläche. Sie betrifft auch Session, Rollenlogik, API-Routen, Datenmodell und Tests. Dasselbe gilt für Dokumentversionen, Statushistorien oder die Frage, ob archivierte Fälle in Berichten weiter auftauchen sollen.

Außerdem war das Portal für mich ein gutes Beispiel dafür, dass Verwaltungssysteme oft weniger durch komplizierte Algorithmen als durch saubere Nachvollziehbarkeit schwierig werden. Ob ein Dokument abgelehnt wurde, wer einen Status geändert hat oder welche Version gerade gilt, muss an mehreren Stellen konsistent bleiben. Dadurch habe ich im Webprojekt noch stärker verstanden, wie wichtig saubere Datenstrukturen, klare Rechte und Tests für spätere Änderungen sind.

Trotz der Unterschiede gab es auch eine klare Gemeinsamkeit. In beiden Projekten reicht es nicht, nur auf die sichtbare Oberfläche zu schauen. Eine Eingabe im Formular oder ein Klick auf eine Lernkarte wirkt erst dann richtig, wenn die Daten im Hintergrund stimmen, Regeln sauber umgesetzt sind und der Zustand der Anwendung nachvollziehbar bleibt. Gerade deshalb waren Themen wie Datenmodellierung, Validierung, Zustandsverwaltung und Tests viel wichtiger, als es von außen vielleicht wirkt.

Für mich war auch hilfreich zu sehen, wie unterschiedlich sich typische Probleme je nach Projektform zeigen. Beim Portal lagen die Schwierigkeiten eher bei Rollen, Rechten, Statuswechseln, Dokumenten und auswertbaren Daten. Bei der App ging es mehr um lokale Persistenz, den eigentlichen Lernablauf, die Auswahl der nächsten Karte und die Frage, wie sich eine mobile Oberfläche auch nach mehreren Sitzungen noch stimmig anfühlt. Dadurch wurde mir klarer, dass Softwareentwicklung zwar oft mit denselben Grundideen arbeitet, die praktische Umsetzung aber je nach Anwendung sehr verschieden aussieht.

Bei der App habe ich noch einmal anders gemerkt, wie eng Oberfläche und Datenlogik zu-

sammenhängen. Ob auf einer Deckkarte drei neue Karten oder keine mehr angezeigt werden, hängt nicht an einem festen Wert im UI, sondern an den Datenbankeinträgen, an den bisherigen Lernprotokollen und an den Regeln für das Tageslimit. Dasselbe gilt für die Statistik, für gemischte Sitzungen und für die Frage, wann eine Karte nach einer Bewertung wieder auftauchen darf. Gerade solche Punkte waren für mich fachlich interessant, weil sie zeigen, dass auch eine kleine App schnell Datenbanklogik, Business-Regeln und Zustandsverwaltung gleichzeitig braucht.

Außerdem habe ich bei der App viel deutlicher gesehen, wie wichtig saubere Überarbeitungen nach der ersten Version sind. Einige Dinge wirken in der fertigen Nutzung recht selbstverständlich, zum Beispiel dass Einstellungen erhalten bleiben, dass deutsche Texte korrekt vorgelesen werden oder dass eine Sitzung sauber endet. Im Code steckt dahinter aber meist mehr Arbeit, weil solche Stellen oft erst mit echten Inhalten, mehreren Sitzungen und kleinen Randfällen auffallen. Das war für mich eine hilfreiche Erfahrung, weil Entwicklung dadurch weniger wie ein einmaliges Bauen und mehr wie ein schrittweises Verbessern wirkt.

Aus dem Studium konnte ich dabei Kenntnisse zu Programmierung, Datenstrukturen, Datenbanken und Softwarearchitektur einbringen. Im Praktikum wurde das aber deutlich greifbarer, weil ich diese Themen nicht isoliert behandelt habe, sondern innerhalb von zwei zusammenhängenden Anwendungen. Gerade die Verbindung aus Build-Konfiguration, Datenhaltung, Oberfläche, Tests und späteren Überarbeitungen war für mich näher an echter Entwicklungsarbeit als vieles, was ich bisher nur aus Lehrveranstaltungen kannte.

III FAZIT UND BEWERTUNG

Aus meiner Sicht hat mir das Praktikum vor allem gezeigt, wie unterschiedlich Entwicklungsarbeit je nach Projekt aussehen kann. Beim Erasmus-Portal stand ein verwaltender Ablauf mit Rollen, Freigaben, Dokumenten und Berichten im Mittelpunkt. Bei der Android-App ging es mehr um lokalen Zustand, Lernlogik und eine einfache Bedienung auf dem Gerät selbst. Beide Projekte hatten also einen anderen Charakter, trotzdem musste ich in beiden Fällen sorgfältig zwischen Oberfläche, Fachlogik und Datenhaltung vermitteln.

Positiv fand ich, dass in beiden Projekten nicht nur einzelne Ansichten entstanden sind, sondern nachvollziehbare Arbeitsstände mit echter Logik dahinter. Im einen Fall sieht man das an Rechten, Statushistorien, Uploads, Filtern, Auswertungen und Tests. Im anderen Fall an Persistenz, Lernlogik, Statistiken, Seed-Daten, Einstellungen und der klaren Trennung der Anwendungsbereiche. Dadurch konnte ich nicht nur sehen, wie Funktionen gebaut werden, sondern auch, wie sie in eine größere Struktur eingebettet werden müssen.

Gerade der Website-Teil hat mir gezeigt, wie stark organisatorische Anforderungen im Code landen. Dinge wie Freigaben, Rollen, Statushistorien, private Dokumente oder Berichte wirken auf den ersten Blick eher verwaltend. In der Umsetzung bedeuten sie aber Datenmodell, serverseitige Prüfungen, geschützte Routen, Tests und viele kleine Entscheidungen zur Nachvollziehbarkeit.

Gerade der App-Teil war für mich ein gutes Beispiel dafür, dass auch ein vergleichsweise kleines Projekt schnell viele Schichten bekommt. Von außen sieht man vor allem Karten, Decks und ein paar Einstellungen. Dahinter liegen aber Datenmodell, Auswahlregeln, lokale Speicherung, Testfälle, Navigation und mehrere Zustände, die gleichzeitig zusammenpassen müssen. Dass sich

diese Teile im Repo Schritt für Schritt entwickelt haben, macht den Arbeitsprozess für mich im Nachhinein gut nachvollziehbar.

Für die Verbindung von Theorie und Praxis war das Praktikum sehr hilfreich. Themen wie Datenmodellierung, SQL-Abfragen, Architekturmuster, Zustandsverwaltung, Build-Konfiguration und Tests waren nicht nur Stoff aus dem Studium, sondern Teil konkreter Entscheidungen im Code. Gleichzeitig sieht man in beiden Projekten auch, dass Software nicht mit der ersten lauffähigen Version abgeschlossen ist. Viele Stellen wurden später noch umgebaut, erweitert oder geglättet. Genau das hat für mich einen realistischen Eindruck von Entwicklungsarbeit vermittelt.

IV ANHANG

- Kopie des Praktikumszeugnisses und ggf. Kopie von Arbeitsproben
- Bestätigung der Teilnahme am Praktikum mit Stempel und Unterschrift der Institution.

V LITERATURVERZEICHNIS

Literaturverzeichnisse lassen sich ebenfalls mit $\text{\LaTeX}$ erstellen, dies ist jedoch eine komplexe Angelegenheit. Eine Möglichkeit besteht in der Einbindung von Literaturdaten in ein $\text{\LaTeX}$ -Dokument über die Pakete „BibTeX“ bzw. „BibLaTeX“. BibTeX ist in fast allen Distributionen von $\text{\LaTeX}$ enthalten. BibLaTeX ist mit einer sehr ausführlichen englischen Dokumentation unter folgendem Link zu finden: <http://www.ctan.org/pkg/biblatex>

Zusätzlich empfiehlt sich die Verwendung einer Literaturverwaltung wie *JabRef*, die ebenfalls frei verfügbar ist und unter diesem Link zum Download bereitsteht: <http://jabref.sourceforge.net/>

Literaturverzeichnisse lassen sich weiterhin auch - wie im folgenden Beispiel - manuell einbinden. Ab der zweiten Zeile erfolgt eine Einrückung um 1cm.

Franck, Norbert / Stry, Joachim (Hrsg.) (2006): Die Technik wissenschaftlichen Arbeitens.
Eine praktische Anleitung. 13. Auflage. Paderborn: Schöningh

VI EIDESSTATTLICHE VERSICHERUNG

Ich versichere eidesstattlich durch eigenhändige Unterschrift, dass ich die Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, habe ich als solche kenntlich gemacht. Die Arbeit ist noch nicht veröffentlicht und ist in gleicher oder ähnlicher Weise noch nicht als Studienleistung zur Anerkennung oder Bewertung vorgelegt worden. Ich weiß, dass bei Abgabe einer falschen Versicherung die Prüfung als nicht bestanden zu gelten hat.

Rostock

(Abgabedatum)

(Vollständige Unterschrift)